Criterion C: Development

Techniques:

- Parameter passing
- Graphics output
- Live audio processing
- Inheritance
    - Calling of methods from other classes

My product is a simple game solution that gives an example of the capabilities of athletic motivation through muscle contraction-detecting video games and doubling as accessibility-friendly ways to control such games.

Parameter passing:
The parameters that I have passed in my game include but are not limited to the calling of and initialization of objects such as the player, obstacles, or move methods within them. For each game tick, the current volume of the audio stream must be read and passed to the ball class to properly move the ball according to spikes in volume. A tick in my program is not a changeable value and is simply one cycle of the main while loop that runs every game action within the main java file. This while loop is stopped when the game ends or is paused. Sensitivity values are passed to the ball class, and these are just some of the implementations of parameter passing.

Graphics output:
My program outputs the movement of the ball and obstacles, inside of a virtual world on JavaFX in real-time, using the sound inputted into the device through the Muscle Spikerbox. This is done in real time by consistently calling “move” methods for the ball and obstacle while running the JavaFX application. The move method uses a variable called dbOver, which is set by a method within the ball class called setStrength, which takes a double variable for a dB value as a parameter.

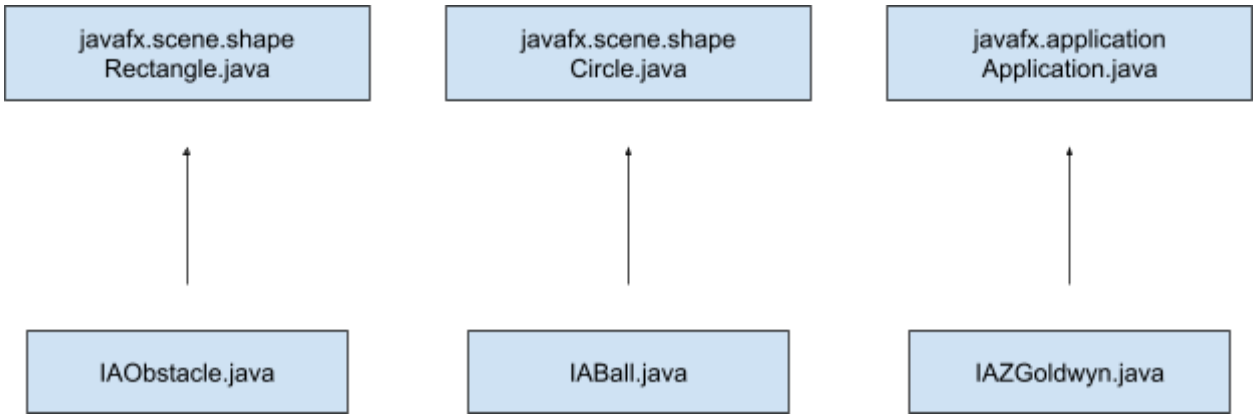
Live audio processing:
My program is a great example of live audio processing, which can also be viewed as external input processing. I depend on the sound input from the device to run my game. Following this text is the code for the processing that I used with many references to the Oracle website about Java Sound. This can be found at
https://www.oracle.com/technetwork/java/javase/sound-dev-guide-1-159173.pdf

```java
// set up audio format
AudioFormat audioFormat = new AudioFormat( sampleRate: 44100,  sampleSizeInBits: 16,  channels: 1,  signed: true,  bigEndian: false);

// get default microphone input
DataLine.Info dataLineInfo = new DataLine.Info(TargetDataLine.class, audioFormat);
TargetDataLine targetDataLine = (TargetDataLine) AudioSystem.getLine(dataLineInfo);

// open microphone to capture audio
targetDataLine.open(audioFormat);
targetDataLine.start();

// start separate thread for audio processing
Thread audioThread = new Thread(() -> {
    // buffer for audio data
    byte[] buffer = new byte[1024];
    ByteArrayOutputStream byteArrayOutputStream = new ByteArrayOutputStream();

//https://docs.oracle.com/javase/tutorial/sound/capturing.html source

    System.out.println("listening... \npress Ctrl+C to exit or close Muscular Movement.");
```

code not included exists in between these images

```java
        byteArrayOutputStream.write(buffer,  off: 0, bytesRead);
        // convert bytes to doubles & calculate dB level
        byte[] audioData = byteArrayOutputStream.toByteArray();
        double sum = 0;
        for (byte audioByte : audioData) {
            double audioSample = audioByte / 32768.0; // convert a byte to a double between -1 and 1
            sum += audioSample * audioSample;
        }
        double rms = Math.sqrt(sum / audioData.length);
        double dB = 20 * Math.log10(rms); //dB math
        // update JavaFX application:
        Platform.runLater(() -> {
            if (countContainer[0] == 5) {//averages a total of 5 audio samples and then executes(this takes milliseconds)
                sdrdb.setValue((avgdBContainer[0] / 5)+ 55.5); //dB Value
                //System.out.println((avgdBContainer[0] / 5)+ 55.5);
                if (( ( (avgdBContainer[0] / 5)  + 55.5) - startGameQuietValue[0]) >= player.getSensitivity()){ //if dB value - silence > sensitivity
                    player.setStrength(( (avgdBContainer[0] / 5)  + 55.5) - startGameQuietValue[0]);//tell the ball how many dB
                }
                countContainer[0] = 1;
                avgdBContainer[0] = 0;
            }
            else {
                avgdBContainer[0] += dB;
                countContainer[0]++;
            }
            btnConfigMax.setText("Click to Set Maximum:  " + (int) sdrConfigDBMax.getValue() + " ");
            btnConfigMin.setText("Click to Set Minimum:  " + (int) sdrConfigDBMin.getValue() + " ");
            btnSetSens.setText("Set Sensitivity: "+ (int) sdrSetSens.getValue());
            lblScore.setText("Score: " + (int)score[0]);
        });
        byteArrayOutputStream.reset();//reset audiostream bytes
    }

});
audioThread.setDaemon(true); // allow the application to exit even if thread is running
audioThread.start();
```

Inheritance & calling of methods from other classes:
In my program, I utilized many different classes. I had a main file that referenced a ball class and an obstacle class. Each of these classes which were coded in separate files created objects that were used within the JavaFX application. This also outlines the ideas of inheritance as the ball and obstacle classes extended shape classes already implemented into JavaFX.scene.shape. This is demonstrated by the flowchart below:



Word Count: 418